IT4931 / IT5384 / IT4043E · SOICT · HUST

Big Data Storage and Processing

Lecture 1 — Overview

Scale · 5V · four-layer stack · storage & processing challenges · applications

Learning outcomes (CLO1)

By the end of this lecture, you should be able to:

1. **Explain** the course definition of big data (distributed storage / processing)
2. **Describe** drivers of data growth and the **5V** model with concrete examples
3. **State** storage requirements: scalability, accessibility, transparency, availability
4. **Contrast** integration & processing challenges (batch vs stream; vs classical HPC)
5. **Sketch** the **four-layer** stack and map later lectures onto it
6. **Illustrate** applications across domains — and why viz, privacy, and data quality matter

Roadmap today

1. **1.1 Introduction** — growth, paradigms, definition, 5V, sources, four-layer stack
2. **1.2 Storage challenges** — SATA properties, scale-out, I/O, landscape
3. **1.3 Integration & processing** — pipelines, batch/stream, analytics preview
4. **1.4 Utilization · quality · privacy** — viz, security, cleaning cost
5. **1.5 Applications** — domains, DS process, course roadmap (L02–L14)

1.1 Introduction to Big Data

From 2008 to 2026

2008–2015 — Rise of “Big Data”

- Hadoop, MapReduce
- 3V / 5V popularized
- Batch under warehouse stress

2015–2020 — Scale & stream

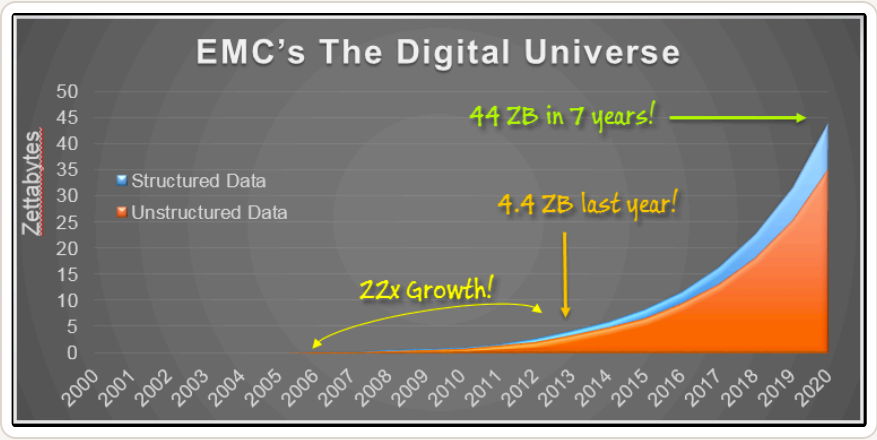
- Spark as default analytics engine
- Kafka + stream pipelines
- Cloud data lakes (S3, ADLS, GCS)

2020–2026 — Real-time & AI

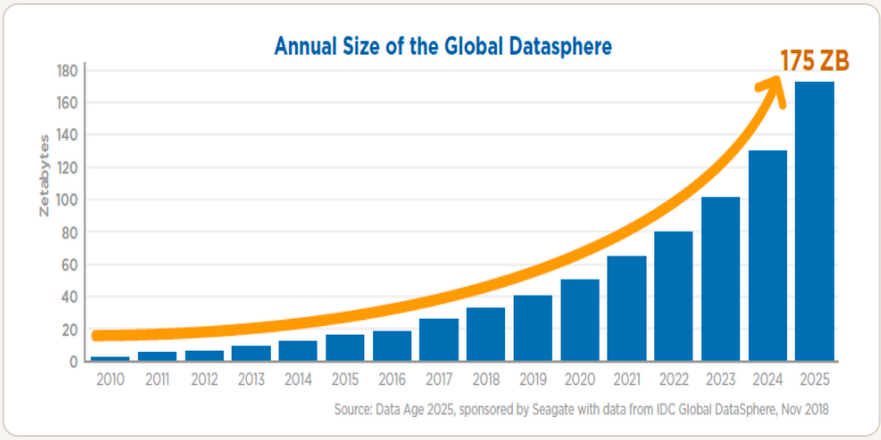
- Lakehouse, low-latency decisions
- AI/ML training & feature pipelines
- Governance, quality, data products
- Language shift: **data at scale**

Storage & processing at scale stay foundational; platforms and use cases evolve.

How large is “large”? (context figures)



Growth context (industry scale)



Scale of data growth toward the mid-2020s

Treat numbers as **order of magnitude**, not facts to memorize — the point is continuous exponential pressure on storage & pipelines.

Visualizing scale

Intuition aids: what does a petabyte / exabyte “feel like” in practice?



Data science as a 4th paradigm

Experimental → theoretical → computational → **data-intensive** science.

http://www.wired.com/wired/issue/16-07 September 2008



“Big data today” — still relevant?

Buzzword faded; **engineering problem** remains: store, move, and compute under SLAs.



What this course means by “Big Data”

Working definition

> Data that needs **distributed storage** and/or **distributed processing** to store, process, or query effectively — with scale, fault tolerance, and acceptable latency.

- Not **only** volume
- Also velocity, variety, veracity, value
- Today: right data, right time, right quality (esp. AI/ML)

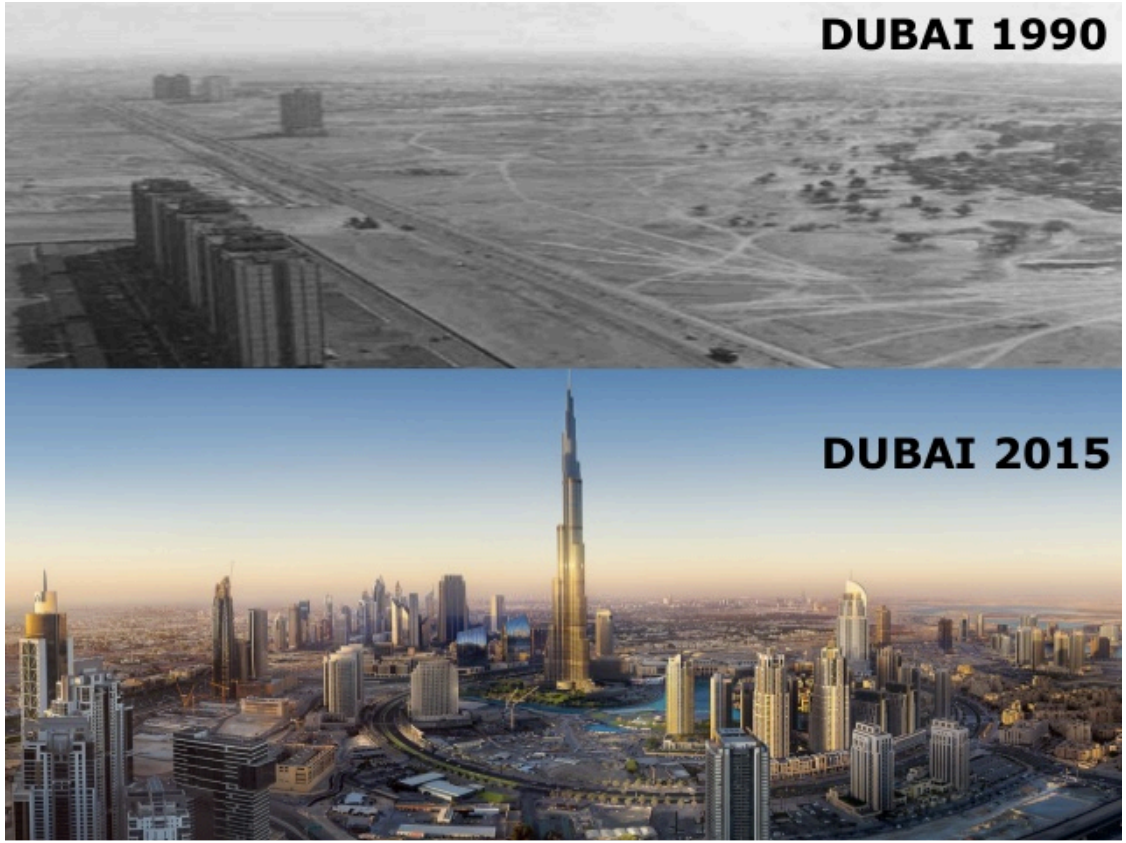
Classic wording

Too large or complex for pre-2006 platforms.

Metaphor

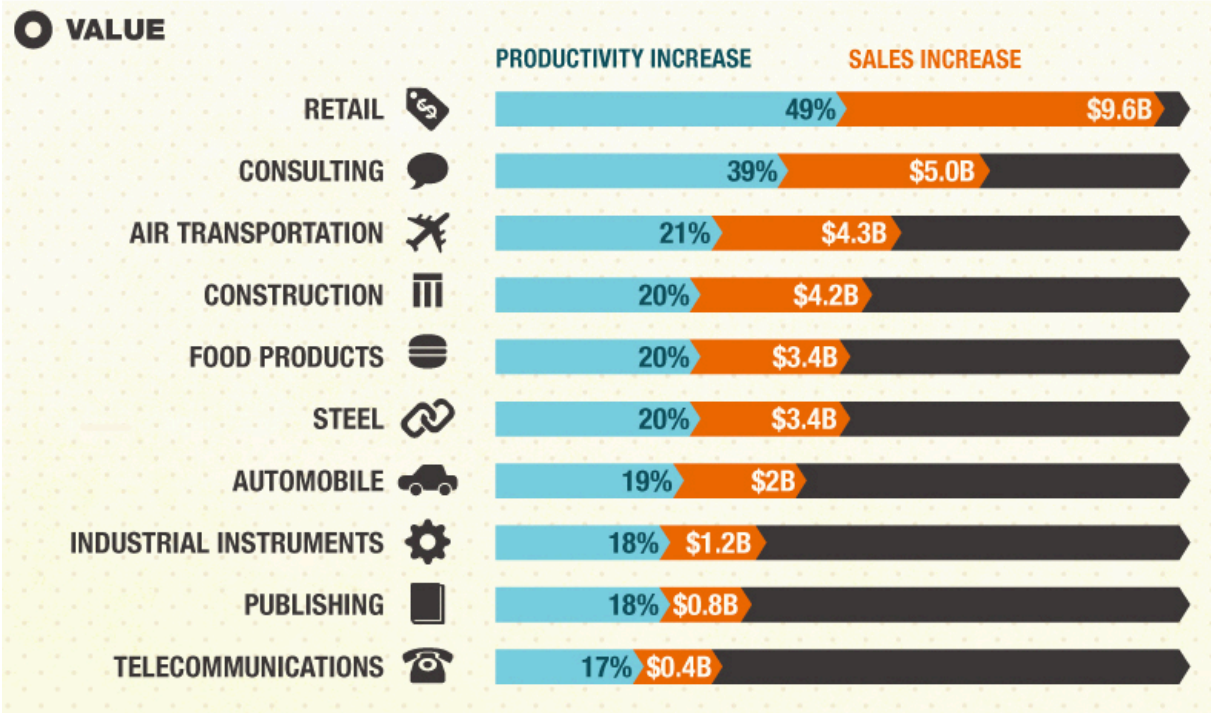
Data as *new oil* — store → refine → extract value.

Data as a strategic resource



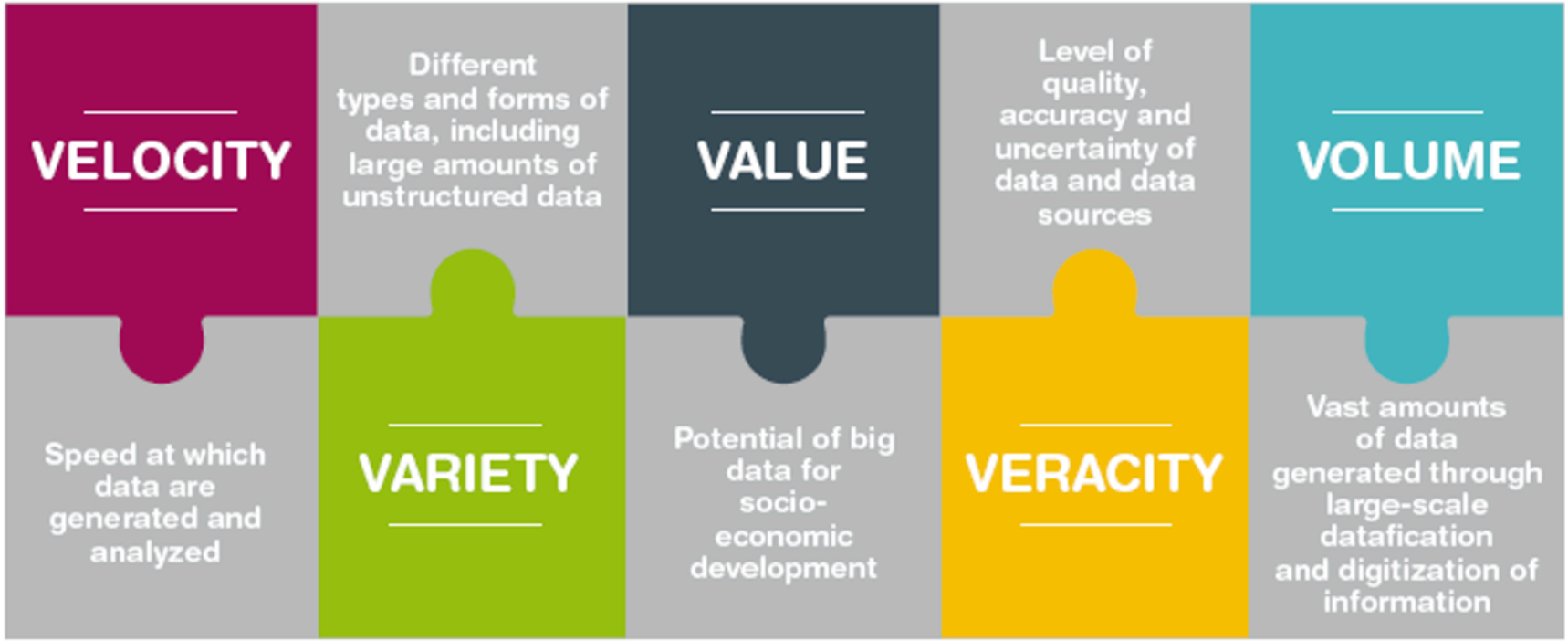
Why value is hard-won

Value is not automatic — collection, cleaning, models, and decisions must connect.



The 5V model

Volume · Velocity · Variety · Veracity · Value — shared vocabulary for this course.



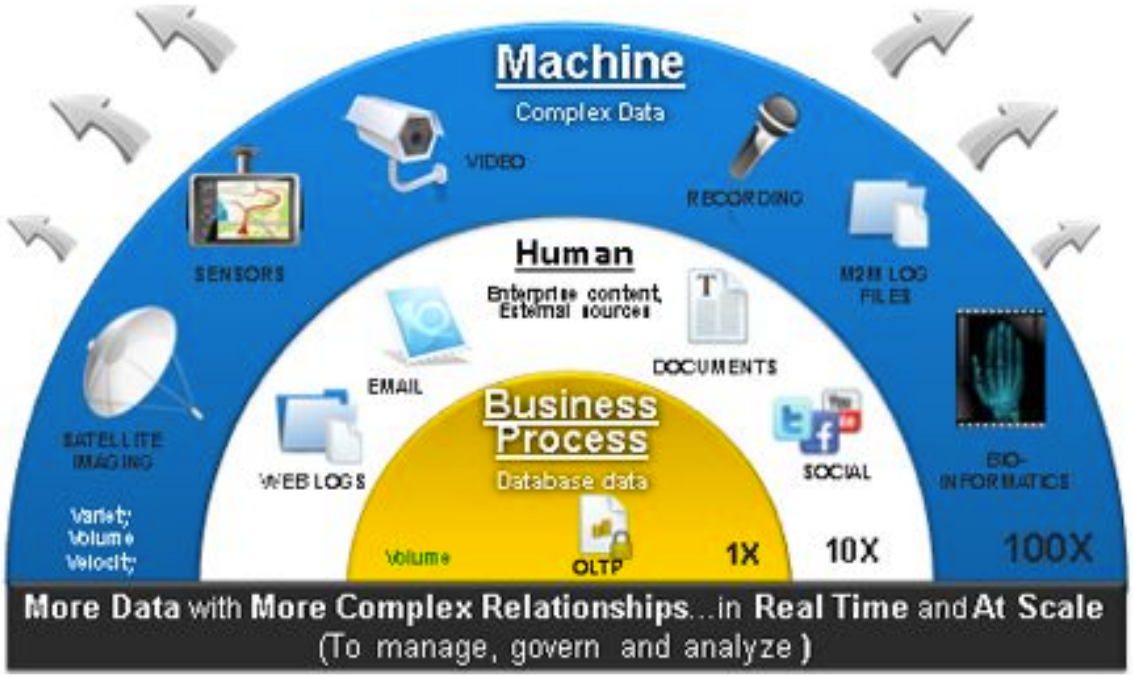
5V — concrete examples

V	Example	System implication
Volume	CERN LHC ~15 PB/year; huge catalogs	Distributed storage, partitioning
Velocity	Millions of events/s	Streams, backpressure, low-latency paths
Variety	Tables, JSON/logs, images/text	Multiple models & file formats
Veracity	Noise, bias, missing fields	Validation, governance
Value	Fraud score, recommendation, alert	Analytics / ML / products

This course emphasizes **storage + integration/processing** that address the first three V's.

Sources of big data

E-commerce, social, IoT, scientific experiments — continuous multi-source generation.



Drivers of the big-data era

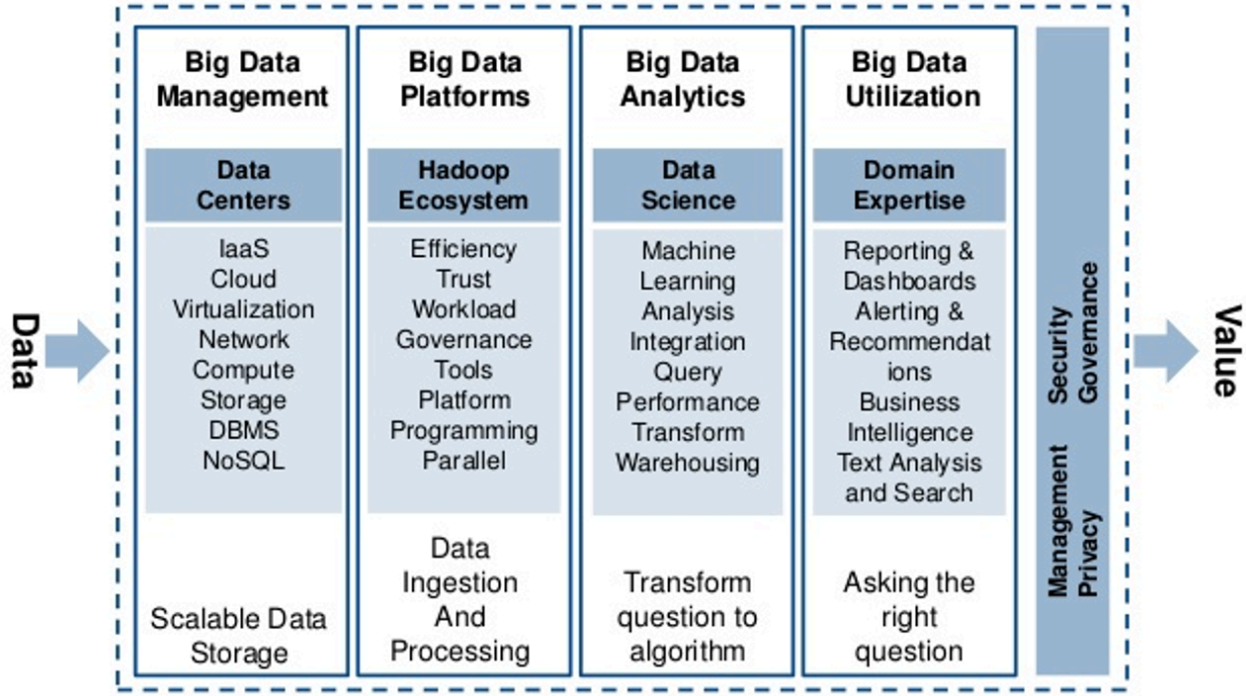
- **Online services:** shopping, social, search, ads — continuous behavioral trails
- **Smart devices & IoT:** phones, wearables, industrial sensors — high-velocity streams
- **Scientific experiments:** bioinformatics, physics, astronomy — extreme volume/variety
- **Enterprise & edge:** logs, CRM/ERP, edge processing for latency-sensitive paths

Also rising: **governance / lineage**, **privacy**, and **energy cost** of keeping everything forever.

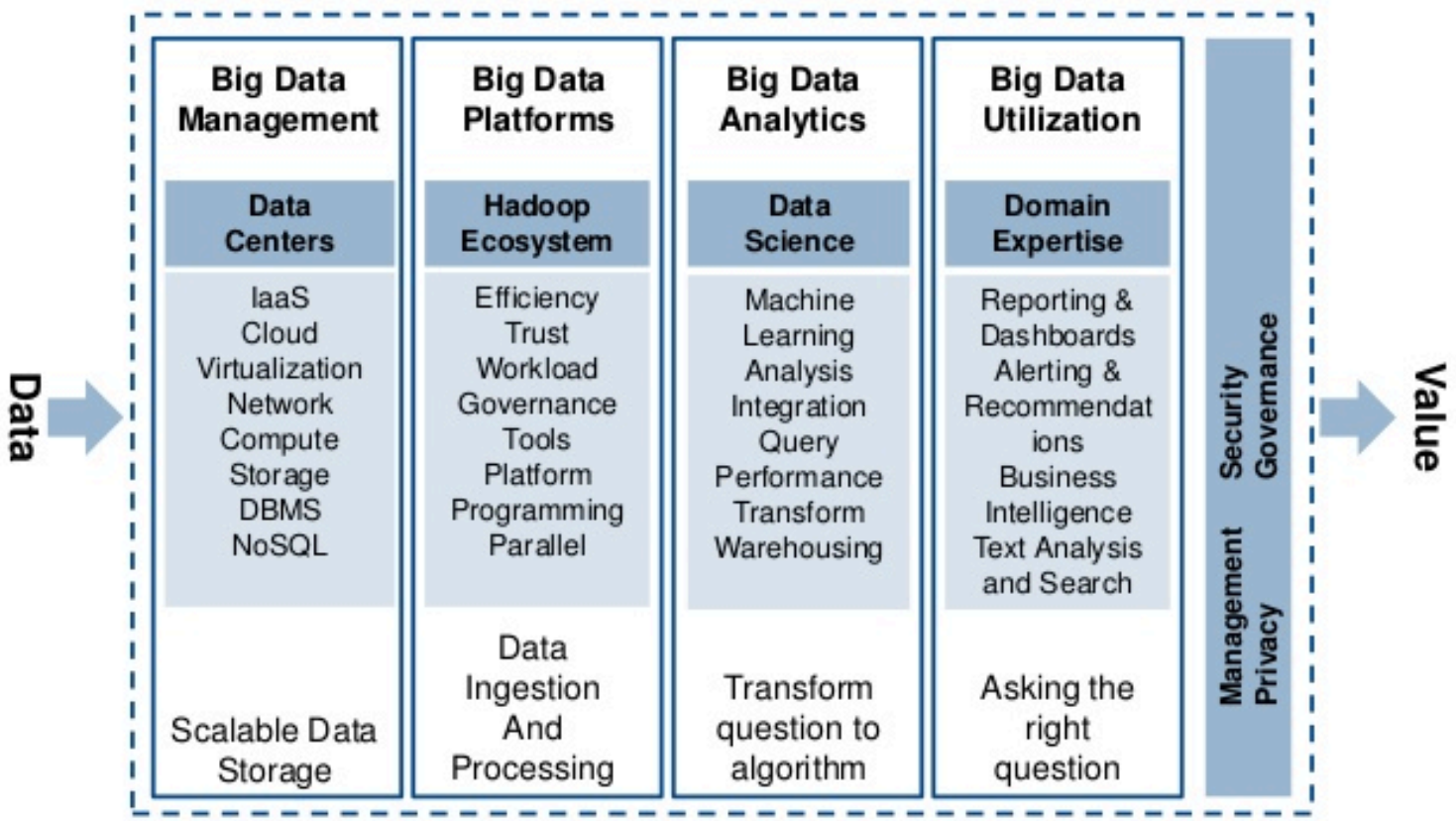
Four-layer technology stack

Layered stack

Storage → Integration & processing → Analytics → Utilization



Layered stack (alternate view)



Layers ↔ this course

Layer	Role	Later lectures
1 Storage	Data at rest, formats, durability	L03 HDFS, L04 lakehouse, L05–L07 NoSQL/DynamoDB/Presto
2 Integration & processing	Pipelines, batch/stream, architectures	L08 Kafka, L09 arch, L10–L12 Spark/Flink
3 Analytics	SQL / stats / ML workloads	L13 Spark ML
4 Utilization	Dashboards, decisions, products	Context for “why” · capped in L14 pipeline

Cross-cutting: **governance, security, quality**.

1.2 Storage challenges

Storage must be scalable (SATA)

Four properties of big-data storage:

Property	Meaning
Scalability	Grow with data over time (prefer **horizontal** add-nodes)
Accessibility	Efficient read/write I/O as size grows
Transparency	Location / distribution hidden behind a simple API
Availability	Survive node/rack failures; serve many concurrent users

Scale-up vs scale-out

Scale-up (vertical)

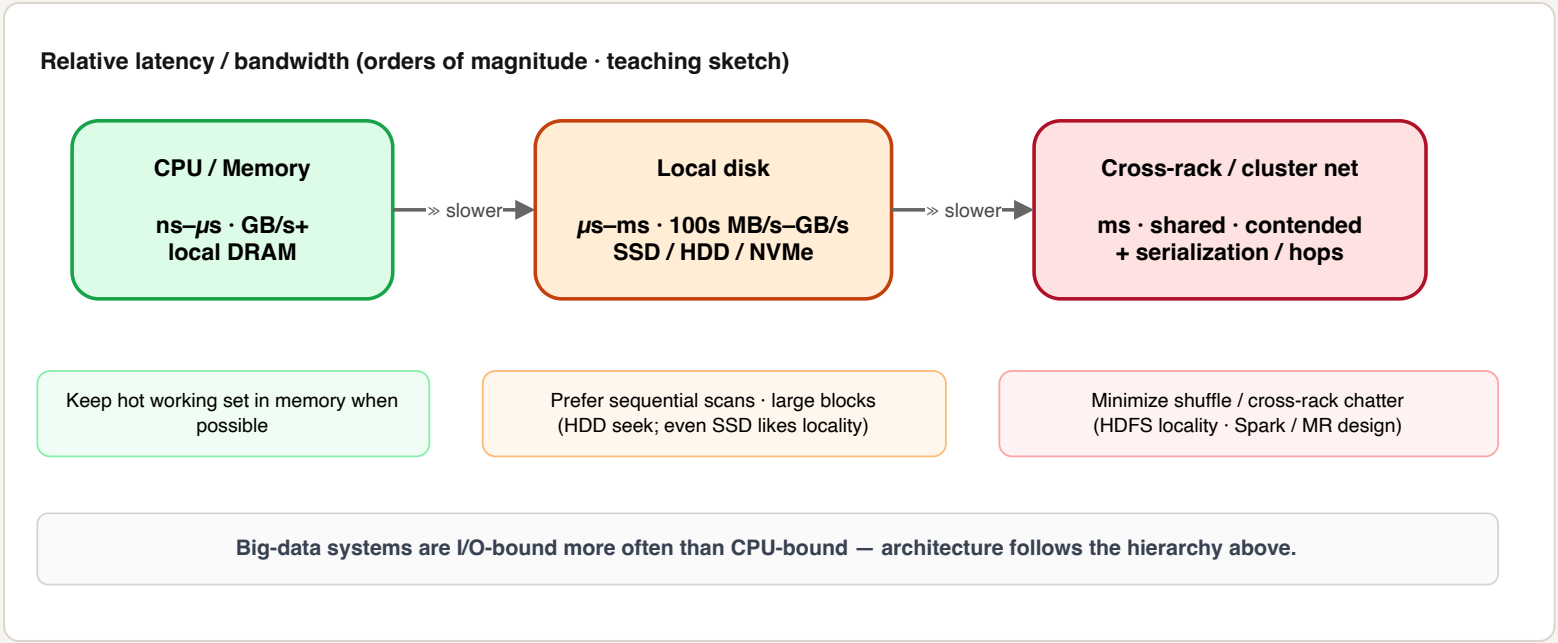
- Bigger box: CPU/RAM/disk
- Simple programming model
- **Limits:** cost curve, single failure domain, hard ceiling

Scale-out (horizontal)

- Many commodity nodes
- Partition + replicate
- **Pays:** network, consistency, ops
- **Wins:** capacity & throughput with cluster size

Big-data systems assume **scale-out** as the default.

I/O reality check



Design implication: locality + sequential scans beat clever CPU tricks when the cluster is I/O-bound (HDFS / MapReduce / Spark shuffle).

Why storage is hard (checklist)

- **Scale & cost:** PB growth, hot/cold tiering, lifecycle policies
- **Durability & availability:** replication, failure domains, rack awareness
- **Access patterns:** sequential analytics vs random serving
- **Models & formats:** files, objects, wide-column, docs; Parquet/ORC/…
- **Security:** encryption, access control across tenants

Storage landscape (preview)

Direction	Idea	Later
Distributed FS	Blocks, replication, rack awareness	**L03 HDFS**
NoSQL / NewSQL	Alternative models & consistency	**L05**
Object storage	S3/ADLS/GCS as lake foundation	Cloud deployments
Lakehouse / tables	Open table formats on files	**L04**

1.3 Integration & processing

Integration challenges

1. **Many sources** — OLTP, app logs, IoT, third-party APIs; uneven rates & interfaces
2. **Many formats** — text/JSON/XML, images/video, domain binaries; schema-on-read vs write
3. **Many schemas/models** — mapping, evolution, schema registry
4. **Security & privacy** — PII, encryption, anonymization, DLP across the pipeline

Mini example: clickstream JSON on Kafka + CDC from OLTP → normalize → lakehouse, handling late/out-of-order events.

Processing challenges

Batch

- Bounded windows, SLA
- Locality-aware scheduling
- MapReduce → **Spark** (L08)

Stream

- Low latency, watermarks
- At-least / exactly-once
- Backpressure & elastic scale
- **Kafka** + stream engines (L06, L09)

Classical OpenMP/MPI: powerful but steep learning curve, weaker commodity fault tolerance, costly HPC — hence Spark/Flink-style engines for many data platforms.

Batch vs stream — one event type

Need	Style	Sketch
Daily revenue report	Batch	Land events → nightly Spark → dashboard
Block fraud in seconds	Stream	Kafka → streaming job → action API
Train recommender weekly	Batch (+ store)	Features from lake → ML (L13)
Online features	Stream + store	Feature updates with low lag

Same raw events; **SLA** chooses batch vs stream (or **Lambda/Kappa** — L09).

Scalable analytics (preview)

Two strategies when data/models outgrow one machine:

1. **Shrink the data** — sub-sampling, PCA, feature extraction/selection
2. **Distribute the algorithm** — e.g. MapReduce k-NN, divide-and-conquer SVM

Curse of dimensionality: more features ⇒ exponentially more samples needed — fixed sample size ⇒ accuracy can drop.

Depth in L10; here only the motivation for distributed compute.

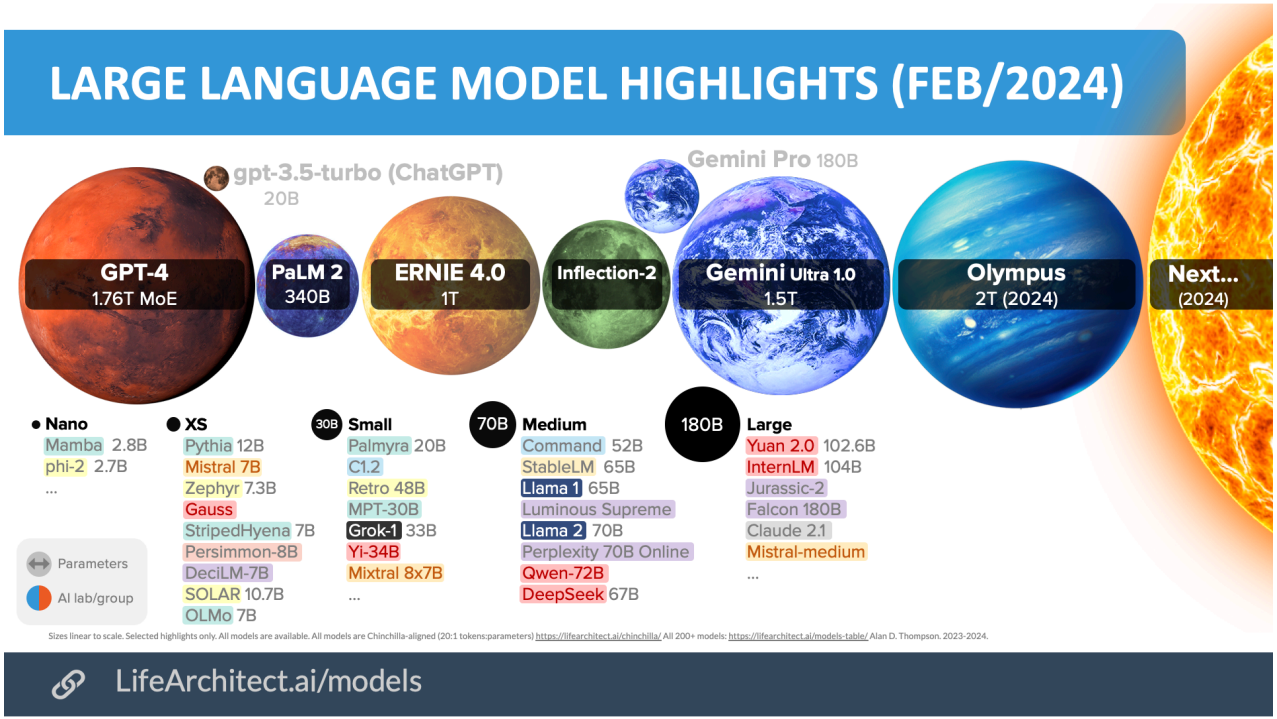
2025–2026 demand drivers

- Real-time analytics & decisions
- AI/ML data pipelines (train, features, monitor)
- Lakehouse: one platform for BI + ML
- Governance, quality, lineage, cost/energy

Course answer: solid storage, messaging, batch/stream compute, lakehouse early (L04), then capstone pipeline with dbt (L14).

AI pressure on data

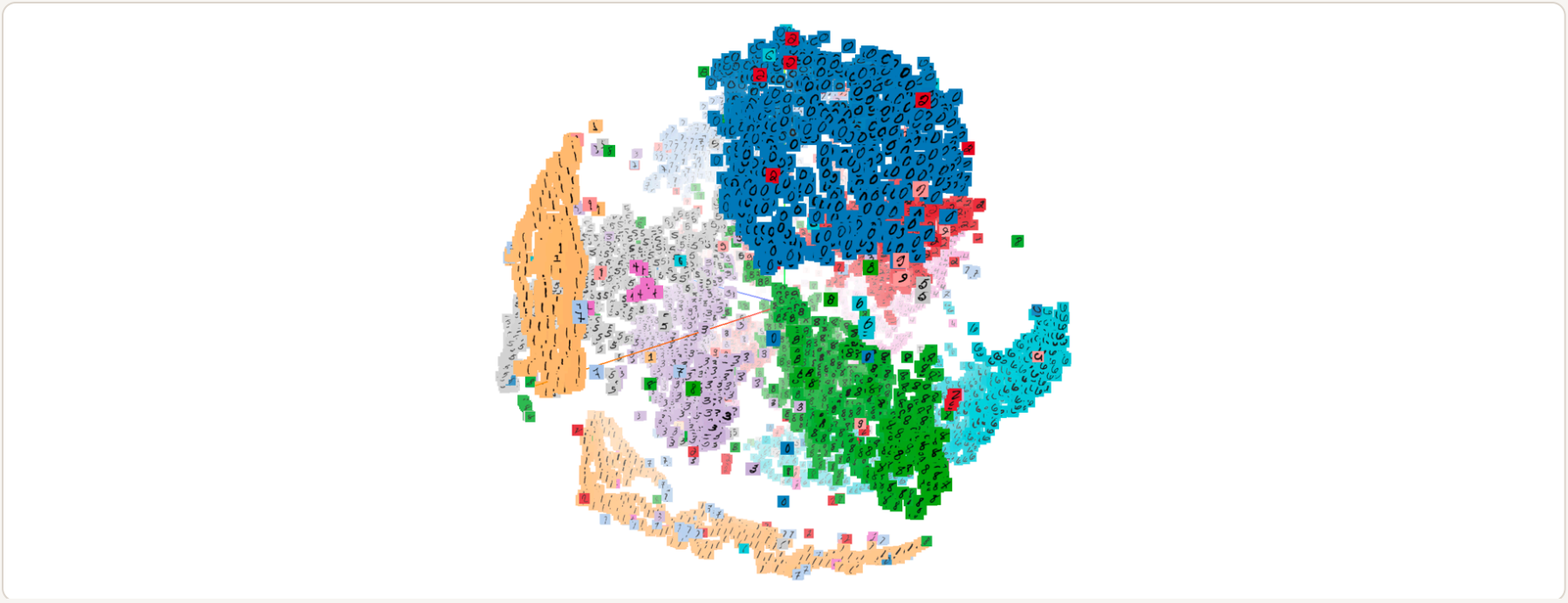
Training data is a bottleneck — pipelines, quality, and governance become first-class.



1.4 Utilization · quality · privacy

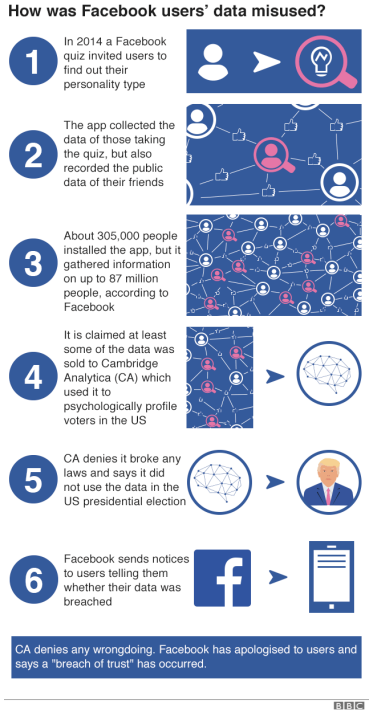
Visualization is not free

Domain expertise + interactive tools + storytelling — at million-row scale you need aggregation/sampling.



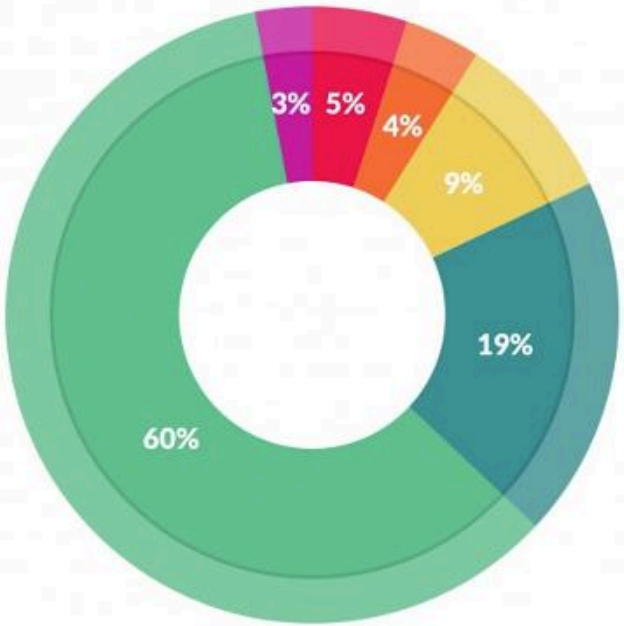
Security & privacy

Access control, anonymization, differential privacy, compliance (GDPR/HIPAA-style constraints).



Most of the work is cleaning

Classic claim: a large share of DS effort is data prep — storage & pipelines must support quality work.



What data scientists spend the most time doing

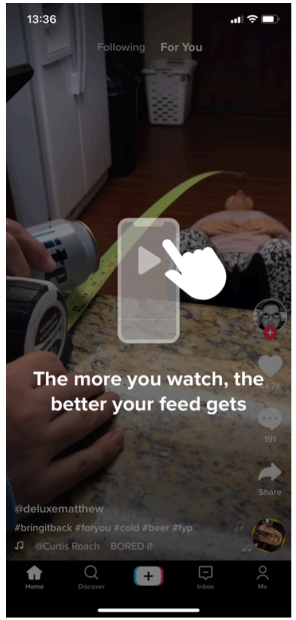
- *Building training sets: 3%*
- *Cleaning and organizing data: 60%*
- *Collecting data sets; 19%*
- *Mining data for patterns: 9%*
- *Refining algorithms: 4%*
- *Other: 5%*

1.5 Popular applications

Industry examples (commerce / social)



Recommendations, logistics, catalog at scale



High-velocity engagement / ranking / ads

Education & healthcare



Personalized paths, assessment, content



Cost, outbreak response, early risk

Government & media



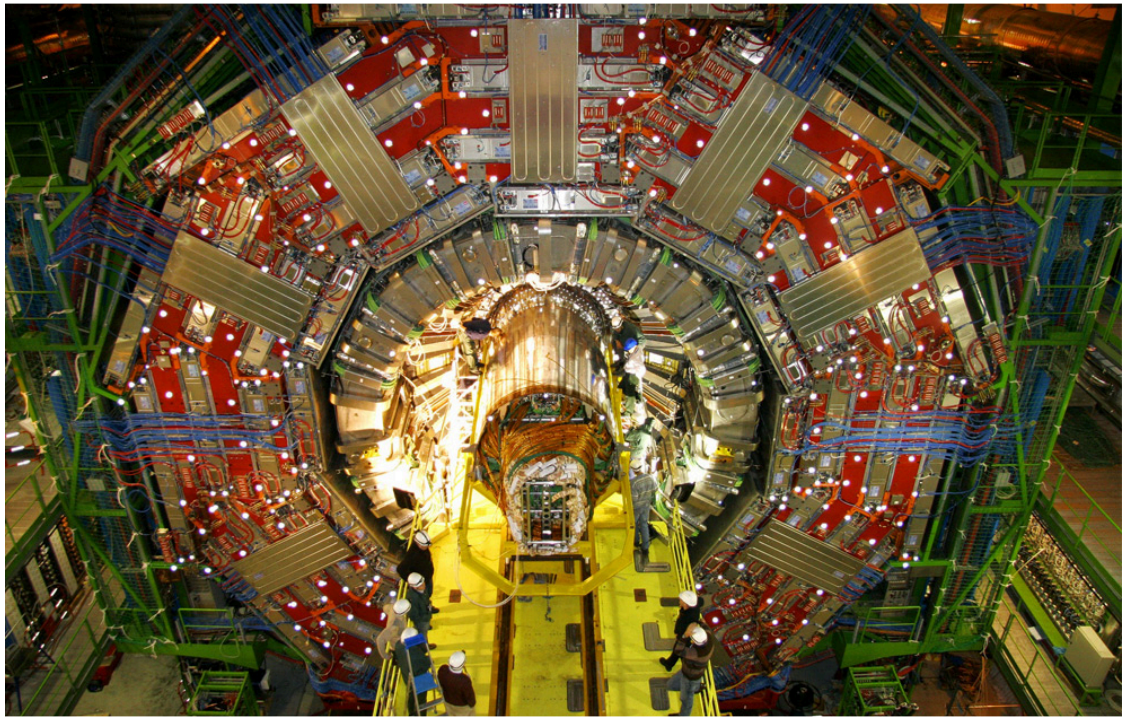
Welfare, safety, fraud, cyber



Taste prediction, personalization (Spotify/Netflix-class)

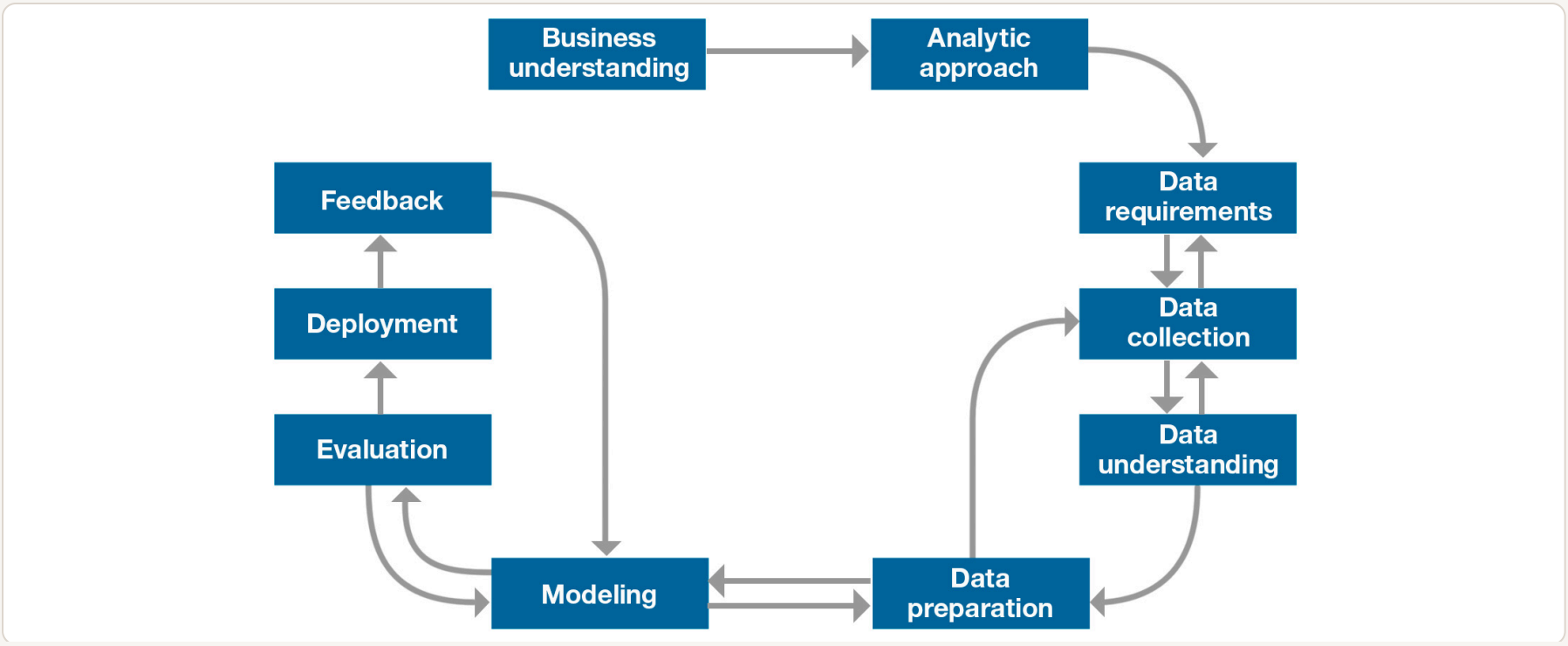
Scientific discovery

CERN LHC-class experiments: ~15 PB/year — storage + processing as science infrastructure.



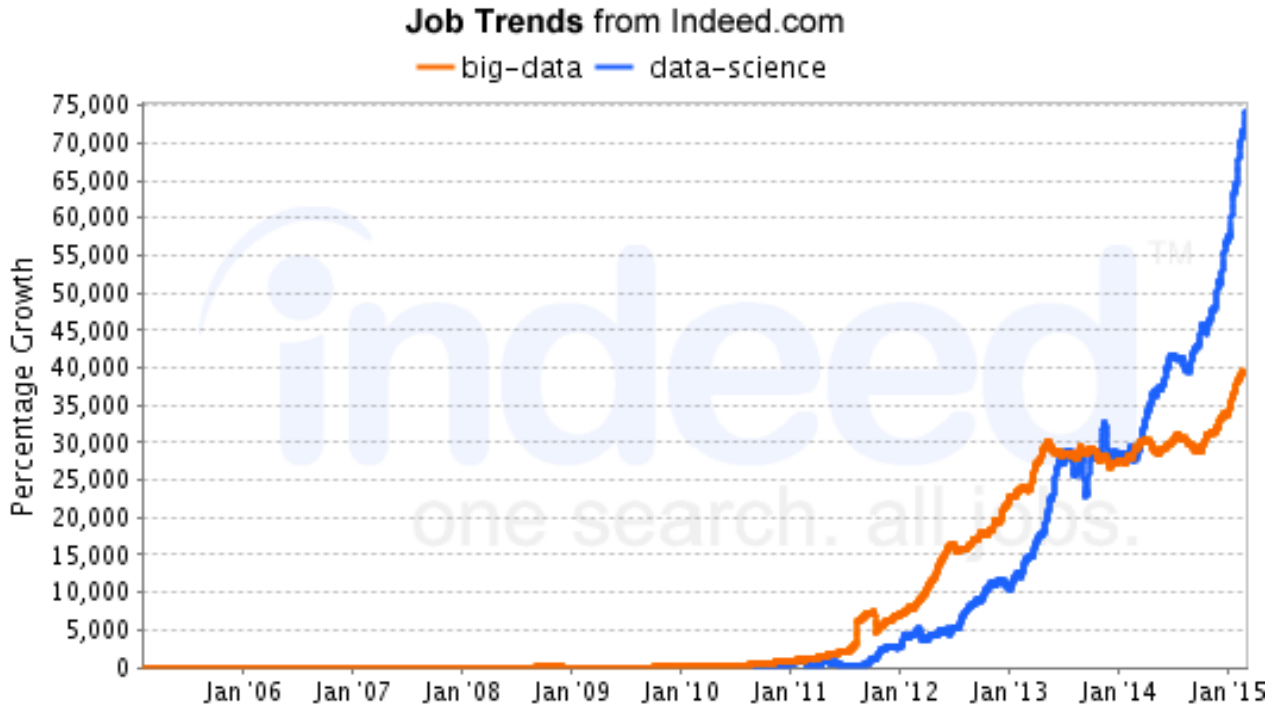
From question to product

IBM-style loop: question → gather → analyze → product — our stack sits under every step.



Skills landscape (optional context)

Roles span engineering, analysis, and domain — this course builds the **storage/processing** core.



Course roadmap (after L01)

L	Topic
L02	Hadoop ecosystem
L03	HDFS
L04	Lakehouse
L05	NoSQL
L06	DynamoDB
L07	Presto / Trino
L08	Kafka

L	Topic
L09	Architectures (Λ / K)
L10	Spark batch
L11	Spark Structured Streaming
L12	Flink internals
L13	Spark ML
L14	Lakehouse pipeline

Storage → messaging → batch/stream → ML → **L14 capstone** (Kafka · Iceberg · dbt · GE · Airflow).

Takeaways

1. Big data **here** = need for **distributed storage/processing**
2. **5V** is shared vocabulary; design follows Volume/Velocity/Variety
3. Storage goals: **scalability, accessibility, transparency, availability**
4. Integration/processing: heterogeneous sources + **batch vs stream** SLAs
5. Remember the **four layers** — this course is strongest on layers 1–2
6. Applications & AI raise the bar on **quality, privacy, and pipelines**

Further reading

- White — *Hadoop: The Definitive Guide*
- Chambers & Zaharia — *Spark: The Definitive Guide*
- Gandomi & Haider — “Beyond the hype…” (IJIM 2015)
- Course textbook / lecture notes — Chapter 1
- Surveys on data-intensive science and the 5V model

Next

L02 — Hadoop ecosystem